第 6 章：控制流

一句话先抓住本章：**Python 控制流的重点不是手工维护下标，而是直接说明要判断什么、遍历什么、何时跳过，以及如何产生结果。**

这些能力会出现在几乎所有真实业务中：按订单状态选择处理路径、批量校验输入、重试外部请求、搜索第一个匹配项，以及从原始数据构造新集合。Python 和 Java 解决的是同一类问题，但 Python 更依赖缩进、对象的真值和可迭代协议来表达流程。

学完本章后，你应该能够：

- 用 `if / elif / else` 写出边界清楚的互斥分支；
- 理解真值、短路求值，以及 `and / or` 不一定返回布尔值；
- 根据数据来源选择 `for`、`range`、`enumerate`、`zip` 或 `while`；
- 分清 `break`、`continue`、`pass` 和循环 `else`；
- 把简单的映射、过滤写成推导式，同时知道何时应保留普通循环；
- 识别 Java 工程师迁移时最容易遇到的边界错误。

先建立 Java → Python 心智模型

要表达的意图	Java 常见写法	Python 常见写法	不能直接等同的地方
选择一个分支	`if / else if / else`	`if / elif / else`	Python 用缩进界定代码块，条件也不要求必须是 `bool`
遍历元素	增强 `for`	`for item in iterable`	Python 面向可迭代对象，不要求有下标或 `size()`
按整数计数	C 风格 `for`	`for number in range(...)`	Python 没有 `for (初始化; 条件; 更新)` 这种语法
同时取得序号和值	手工维护 `int i`	`enumerate(iterable)`	`start` 改变的是产生的编号，不是原容器下标
并行遍历两组值	下标 + 长度检查	`zip(a, b, strict=True)`	默认 `zip` 会按最短输入静默停止

条件未知次数的循环	`while` / `do-while`	`while`	Python 没有内置 `do-while`
简单过滤与转换	Stream 或循环 + `add`	推导式	列表推导式立即创建列表， 不等同于惰性的 Java Stream
遍历完仍未找到	标志变量或提前 `return`	提前 `return` 或循环 `else`	循环 `else` 的条件是“没有被 `break` 终止”

先记住三个选择问题：

1. 只是在一批数据中逐个处理吗？优先 `for` 。
2. 退出由不断变化的状态决定吗？使用 `while` ，并明确终止条件。
3. 只是把输入逐个转换或过滤成新集合吗？逻辑足够简单时使用推导式。

6.1 分支怎样决定哪段代码执行

缩进就是代码块边界

Java 用 `{}` 包围分支，Python 在条件后写 `:` ，再用一致的缩进表示从属关系。惯例是每层使用 4 个空格：

```
temperature = 29

if temperature >= 35:
    feeling = "高温预警"
elif temperature >= 26:
    feeling = "偏热"
elif temperature >= 18:
    feeling = "舒适"
else:
    feeling = "偏冷"
```

Python 从上到下检查条件，只执行第一个真值分支。 `29` 命中 `temperature >= 26` 后，不会再检查 `>= 18` 。因此多个区间有包含关系时，要先写更严格、更具体的条件；如果先写 `temperature >= 18` ，高温数据也会提前进入“舒适”分支。

`if` 、 `for` 和 `while` 不会像 Java 的 `{}` 那样创建新的块级作用域。分支里绑定的名字仍属于当前模块或函数作用域，但这不代表它一定存在：

f7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-bb8147178d5fab4c8006c303628.html

关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代...

```
def order_status(paid: bool) -> str:
    if paid:
        status = "已支付"
    return status  # paid 为 False 时，status 从未绑定，会抛 UnboundLocalError
```

如果分支后的代码一定要读取某个名字，就让所有路径都为它赋值，或者在分支前提供合理初值。

分支顺序本身就是业务规则

互斥分类通常从异常边界到正常范围，或者从最具体条件到最宽泛条件：

```
def classify_score(score: int) -> str:
    if not 0 <= score <= 100:
        return "非法分数"
    if score >= 90:
        return "优秀"
    if score >= 60:
        return "及格"
    return "不及格"
```

每个命中分支都立即 `return`，因此不必再写 `elif`。这种写法可以先排除非法状态，让后面的主路径保持平坦。它和 Java 的 guard clause（守卫式提前返回）思路相同。

Python 条件使用对象的真值

Java 的 `if` 条件必须产生 `boolean`。Python 会询问对象“放在条件位置时算真还是假”，这叫**真值判断**。

常见假值如下：

类别	假值示例
空值	`None`
布尔值	`False`
数字零	`0` 、 `0.0`
空字符串	`""`
空容器	`[]` 、 `()` 、 `{}` 、 `set()`

/lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-bb8147178d5fab4c8006c303628.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下

其他大多数对象都是真值。因此检查容器是否非空时，通常直接写：

```
items = []

if items:
    print("有数据")

if not items:
    print("没有数据")
```

这种简写只有在业务语义也允许“所有假值等价”时才安全。 `None` 、 `0` 和空字符串可能代表三种不同状态：

```
discount: int | None = 0

if discount is None:
    print("调用者没有提供折扣")
elif discount == 0:
    print("调用者明确指定不打折")
```

`None` 是单例，使用 `is None` 或 `is not None` 判断身份；普通值比较使用 `==` 。

`and` 与 `or` 既会短路，也会返回操作数

短路规则和 Java 相似：

- `a and b` ：如果 `a` 已是假值，就不计算 `b` ；
- `a or b` ：如果 `a` 已是真值，就不计算 `b` ；
- `not value` ：把对象的真值反转成真正的 `bool` 。

```
user = None

if user is not None and user.get("name"):
    print(user["name"])
```

当左侧为假时， `user.get(...)` 不会执行，因此不会对 `None` 调用方法。保护条件必须放在可能出错的表达式前面。

与 Java 的 `&&` / `||` 不同，Python 的 `and` / `or` 返回决定结果的那个操作数，而不一定返回 `True` 或 `False` ：

```
typed_name = ""
```

f7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-bb8147178d5fab4c8006c303628.html

|关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商

```
typed_name = ""
display_name = typed_name or "访客"  # 结果是字符串 "访客"
```

这适合提供“假值即缺省”的默认值。如果 `0` 、 `False` 或空字符串是合法输入，就不要用 `or` 抹平它们，应显式判断 `is None` 。

区间和二选一可以写得更贴近意图

Python 支持链式比较：

```
age = 20

if 18 <= age < 60:
    print("年龄在目标区间内")
```

它表达的关系相当于 Java 的 `age >= 18 && age < 60` 。链中的中间表达式只求值一次，这不等于先得到 `18 <= age` 的布尔值，再拿布尔值和 `60` 比较。

Java 的三元表达式 `condition ? yes : no` 在 Python 中写成：

```
amount = 120
label = "大额" if amount >= 100 else "普通"
```

阅读顺序是“条件为真时的值 → 条件 → 条件为假时的值”。它只适合简单的二选一；多层嵌套、日志或状态更新应改回普通分支。

6.2 `for` 为什么通常不需要下标

`for` 直接向可迭代对象取值

Python 的 `for` 更接近 Java 增强 `for` ，而不是 C 风格计数循环：

```
languages = ["Java", "Python", "Go"]

for language in languages:
    print(language)
```

列表、元组、字符串、字典、集合、文件、`range` 和生成器都可以提供一串值，因此都属于可迭代对象。循环每轮取得

z72502x6dg8fpjj0sff40000gp/T/crossnote-f-bb8147178d5fab4c8006c303628.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计

列表、元组、字符串、字典、集合、文件、`range` 和生成器都可以提供一串值，因此都属于**可迭代对象**。循环每轮取得一个值，把它绑定到 `language`，然后执行循环体；数据源耗尽后自然结束。

只为取元素而写下标会增加无用状态：

```
# 不推荐：循环真正需要的是元素，而不是下标。
for index in range(len(languages)):
    print(languages[index])
```

需要序号时用 `enumerate`，需要并行取多组值时用 `zip`。只有算法本身依赖位置、相邻元素或原地替换指定位置时，才优先考虑下标。

循环变量是名字绑定，不是源容器的槽位

给循环变量重新赋值，只是让这个名字指向新对象，不会替换列表元素：

```
numbers = [1, 2, 3]

for number in numbers:
    number *= 10

print(numbers)  # [1, 2, 3]
```

要构造转换后的数据，创建新列表通常最清楚。确实要原地改列表时，应按位置赋值，例如 `numbers[index] = ...`。

普通 `for` 也不会创建块级作用域。非空循环结束后，循环变量仍绑定着最后一个值；空数据源则可能让它从未绑定。不要依赖循环外的循环变量表达结果，应显式保存需要的状态或提前返回。

解包让一条记录直接落到多个名字

当每个元素本身包含多个部分时，`for` 左侧可以直接解包：

```
records = [("Alice", 95), ("Bob", 88)]

for name, score in records:
    print(f"{name}: {score}")
```

这仍是一个循环：每轮先取出一个二元组，再把其中两个值分别绑定到 `name` 和 `score`。元素数量与左侧名字数量不一致时会抛 `ValueError`，不会默默补齐。

lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-bb8147178d5fab4c8006c303628.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计

遍历字典本身默认得到键；同时需要键和值时使用 `.items()`：

```
settings = {"timeout": 3, "retries": 2}

for key, value in settings.items():
    print(key, value)
```

不要在遍历列表时增删元素，也不要在遍历字典时改变键的数量。前者容易跳过数据，后者会触发运行时错误。需要筛选时通常构造新容器；确需删除时则遍历副本或待删除键列表。

`range` 提供整数，但不预先创建整数列表

Python 没有 C 风格 `for`，固定次数或整数区间用 `range`：

写法	产生的整数	说明
`range(4)`	`0, 1, 2, 3`	起点默认为 `0`，步长默认为 `1`
`range(2, 6)`	`2, 3, 4, 5`	包含起点，不包含终点
`range(1, 8, 2)`	`1, 3, 5, 7`	第三个参数是步长
`range(5, 0, -1)`	`5, 4, 3, 2, 1`	反向遍历必须使用负步长

左闭右开的规则与切片一致。`range` 是轻量、惰性的范围对象，不会先把全部整数装进列表：

```
numbers = range(1_000_000)
print(numbers)         # range(0, 1000000)
print(list(range(4)))  # 仅在确实需要列表时转换
```

三个常见边界要单独记住：

- `range(1, 5, -1)` 的方向与步长矛盾，结果为空，不会自动纠正；
- 步长不能是 `0`，否则抛 `ValueError`；
- 无论正向还是反向，`stop` 都不包含在结果中。

6.3 需要序号或并行数据时怎么遍历

`enumerate` 同时产生编号和值

lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-bb8147178d5fab4c8006c303628.html

... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计费

`enumerate` 同时产生编号和值

```
languages = ["Java", "Python", "Go"]

for display_number, language in enumerate(languages, start=1):
    print(f"{display_number}. {language}")
```

`enumerate` 每轮产生一个（编号，元素）二元组，左侧再把它解包。`start=1` 只让编号从 1 开始，适合用户可见的序号；它不会改变原列表，也不意味着第一个元素的真实下标变成了 1。

相比手工执行 `index += 1`，`enumerate` 不会因为某个 `continue` 分支漏掉更新，也可以直接包装没有下标的可迭代对象。

`zip` 按位置配对，长度策略要明确

```
names = ["Alice", "Bob", "Cara"]
scores = [95, 88, 91]

for name, score in zip(names, scores, strict=True):
    print(name, score)
```

`zip` 每轮从各输入中各取一个值，再组成元组。它和 `enumerate` 都是惰性的：创建对象时不会把全部结果算完，只有循环、`list(...)` 等消费者取值时才真正推进。

默认 `zip` 在最短输入耗尽时停止：

```
pairs = list(zip(["Alice", "Bob"], [95]))
print(pairs)  # [('Alice', 95)], Bob 被静默丢弃
```

如果业务要求输入等长，本项目使用的 Python 3.14 可以明确写 `strict=True`：

```
pairs = list(zip(["Alice", "Bob"], [95], strict=True))
# 实际消费到长度不一致的位置时抛 ValueError
```

只创建 `zip(..., strict=True)` 还没有完成校验，因为惰性对象尚未走到不一致的位置。必须真正迭代它，异常才会出现。

序号和多组数据可以组合：

2502x6dg8fpjj0sff40000gp/T/crossnote-f-bb8147178d5fab4c8006c303628.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

序号和多组数据可以组合：

```
for number, (name, score) in enumerate(
    zip(names, scores, strict=True),
    start=1,
):
    print(f"{number}. {name}: {score}")
```

如果多个平行列表长期共同描述同一业务实体，应考虑把字段放进元组、 `dataclass` 或类。 `zip` 适合临时配对，不应代替清晰的数据模型。

6.4 次数不确定时如何安全使用 `while`

`for` 的终点来自数据源耗尽， `while` 的终点来自条件变为假。因此 `while` 适合重试、轮询、等待状态变化等“事先不知道准确次数”的流程：

```
attempt = 0
max_attempts = 3

while attempt < max_attempts:
    attempt += 1
    print(f"第 {attempt} 次尝试")
```

阅读或编写 `while` 时，必须回答三个问题：

1. 初始状态是什么？这里是 `attempt = 0` 。
2. 什么条件允许继续？这里是 `attempt < max_attempts` 。
3. 每轮如何靠近终点？这里是 `attempt += 1` 。

第三点缺失时，循环条件可能永远为真。真实重试通常还需要最大次数、超时、异常分类和取消机制，不能只靠无限循环。

Python 没有内置 `do-while` 。必须至少执行一次时，可以先执行一次再进入 `while` ，或者使用带明确退出条件的 `while True` ：

```
number = 0

while True:
    number += 1
```

dg8fpjj0sff40000gp/T/crossnote-f-bb8147178d5fab4c8006c303628.html

相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

```
number = 0

while True:
    number += 1
    print(number)
    if number >= 3:
        break
```

`while True` 本身不是错误，但读者应能快速找到可靠的 `break`、`return` 或异常退出路径。

6.5 一轮循环和整个循环分别怎样停止

语句或结构	对流程的影响	典型用途
`continue`	跳过本轮余下语句，开始下一轮	排除非法或不相关数据
`break`	立即结束最近一层循环	找到目标后停止搜索
`pass`	什么都不做，然后继续执行下一条语句	暂时提供语法要求的空代码块
循环 `else`	循环未被 `break` 终止时执行	表达“遍历完仍未找到”或“重试已耗尽”

`continue` 适合先排除异常输入

```
valid_scores = []

for score in [95, -1, 82, 101, 60]:
    if not 0 <= score <= 100:
        continue
    valid_scores.append(score)
```

这种守卫式写法先结束无效数据的本轮处理，主路径不需要包在多层 `if` 里。

`pass` 不会跳过本轮

```
for item in ["ready", "", "done"]:
    if not item:
        pass
    print(repr(item))  # 空字符串仍会打印
```

2502x6dg8fpjj0sff40000gp/T/crossnote-f-bb8147178d5fab4c8006c303628.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

`pass` 只是一条合法的空语句。它常用于尚未实现的函数体、类体或分支占位；要跳过本轮后续代码必须使用 `continue`。生产代码里长期保留 `pass`，通常意味着遗漏了处理或留下了无用分支。

循环 `else` 应读成“没有遇到 `break`”

```
products = ["键盘", "鼠标", "显示器"]
target = "音箱"

for index, product in enumerate(products):
    if product == target:
        print("找到了，位置是", index)
        break
else:
    print("遍历完仍未找到")
```

准确规则是：

- 数据源正常耗尽，执行 `else`；
- `while` 的条件正常变为假，执行 `else`；
- 循环中执行过任意次数的 `continue`，最后正常耗尽，仍执行 `else`；
- 通过 `break`、`return` 或异常离开，不执行 `else`。

它不是“循环一次也没执行才进入的分支”。如果团队对这个语法不熟，封装成函数并在找到时提前 `return`，通常也很清楚。

`break` 和 `continue` 只作用于最近一层循环。Python 没有 Java 的带标签 `break outer;`；多层搜索可以提取成函数后提前 `return`，或重新设计数据流，避免用复杂标志位穿透多层循环。

6.6 什么时候用推导式，什么时候保留普通循环

列表推导式表达“遍历 → 过滤 → 转换 → 收集”

```
even_squares = [
    number**2
    for number in range(1, 11)
    if number % 2 == 0
]
```

z72502x6dg8fpjj0sff40000gp/T/crossnote-f-bb8147178d5fab4c8006c303628.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

列表推导式表达“遍历 → 过滤 → 转换 → 收集”

```
even_squares = [
    number**2
    for number in range(1, 11)
    if number % 2 == 0
]
```

不要强迫自己从左到右硬读。按数据流拆解更容易：

1. `for number in range(1, 11)` 依次提供 `1` 到 `10`；
2. `if number % 2 == 0` 丢弃奇数；
3. `number**2` 把保留值转换成平方；
4. 外层 `[...]` 立即创建并返回新列表。

它等价于：

```
even_squares = []

for number in range(1, 11):
    if number % 2 == 0:
        even_squares.append(number**2)
```

可以先把列表推导式类比为简单的 Java Stream `filter` + `map` + `collect`，但不能完全等同：推导式自身不是可复用流水线，方括号版本会立即算完并持有整个结果。

推导式中的两个 `if` 位置表示不同意图：

```
scores = [95, 52, 88]

# for 后的 if 是过滤：不及格的分数不进入结果。
passed = [score for score in scores if score >= 60]

# for 前的 if...else 是转换：每个分数都产生一个标签。
labels = ["及格" if score >= 60 else "不及格" for score in scores]
```

Python 3 的推导式拥有自己的局部作用域，`number` 等临时名字不会泄漏到外部；这与普通 `for` 的循环变量在循环后仍可见不同。

/lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-bb8147178d5fab4c8006c303628.html

t... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

外层符号决定收集结果的类型

```
languages = ["Java", "Python", "JavaScript", "Go"]

initials = {language[0] for language in languages}
parity = {
    number: "偶数" if number % 2 == 0 else "奇数"
    for number in range(1, 6)
}
```

- `[value for ...]` 创建列表，保留遍历顺序和重复值；
- `{value for ...}` 创建集合，自动去重，不应用来表达业务顺序；
- `{key: value for ...}` 创建字典，重复键以后出现的值为准。

空集合仍写作 `set()`，因为 `{}` 表示空字典。

生成器表达式按需产出，而且通常只能消费一次

把方括号换成圆括号，会得到生成器表达式：

```
total = sum(number**2 for number in range(1, 5))
```

它不会先创建完整平方列表。`sum` 每请求一个值，生成器才计算一个值并交出。只需顺序消费一次、数据量较大，或直接传给 `sum / any / all` 时，这种方式避免了中间列表。

生成器被消费后不会自动倒带，也不能像列表一样随机按下标访问。如果需要反复遍历、查看长度或随机访问，应创建列表，或者每次重新创建生成器。

推导式与 `lambda` 不是同一种东西

```
fields = [{"name": "run_id"}, {"name": "phase"}]
names = [item["name"] for item in fields]
getter = lambda item: item["name"]
```

第二行会遍历 `fields` 并立即创建字段名列表。第三行只创建一个匿名函数：以后传入一个 `item` 时，它返回 `item["name"]`；它自己不会遍历任何数据。简单批量转换通常优先推导式，传递一个短小回调规则时才考虑 `lambda`。

出现业务过程时，普通循环更清楚

vf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-bb8147178d5fab4c8006c303628.html

相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计

出现业务过程时，普通循环更清楚

推导式适合无副作用的简单映射和过滤。出现下面任一情况，优先使用普通循环：

- 每个元素需要多步状态更新；
- 需要日志、指标或细分跳过原因；
- 需要 `break` 、 `continue` 或局部异常处理；
- 出现多层 `for` 、多个难懂条件或嵌套条件表达式。

不要用 `[print(order) for order in orders]` 代替循环。它会为了副作用创建一份全是 `None` 的无用列表。Pythonic 的含义是意图清楚，不是行数最少。

6.7 把控制流放进一个订单处理过程

假设每条订单依次包含订单号、金额、是否支付、是否取消。业务要求取消状态优先于金额校验，只有未取消、金额有效且已支付的订单才能完成：

```
orders = [
    ("A100", 120.0, True, False),
    ("A101", -20.0, True, False),
    ("A102", 80.0, False, False),
]
completed = []

for order_id, amount, paid, cancelled in orders:
    if cancelled:
        print(f"{order_id}: 已取消，跳过")
        continue
    if amount <= 0:
        print(f"{order_id}: 金额非法，跳过")
        continue
    if not paid:
        print(f"{order_id}: 尚未支付，跳过")
        continue
    completed.append((order_id, amount))
```

这里的控制流可以按数据变化来理解：

1. 元组解包把一条订单拆成四个有业务含义的名字；

/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-bb8147178d5fab4c8006c303628.html

?相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下4

这里的控制流可以按数据变化来理解：

1. 元组解包把一条订单拆成四个有业务含义的名字；
2. 三个守卫条件按优先级排除订单， `continue` 保证失败订单到此为止；
3. 只有通过所有守卫的订单才能进入 `completed` ；
4. 因为流程需要输出不同失败原因，普通循环比复杂推导式更合适。

过滤完成后，简单的派生结果再交给推导式和生成器表达式：

```
completed_ids = [order_id for order_id, _ in completed]
total_amount = sum(amount for _, amount in completed)
average_amount = total_amount / len(completed) if completed else None
```

`_` 是"这个位置的值当前不使用"的命名惯例，不是特殊关键字。平均值先检查 `completed` 的真值，避免空列表导致除零错误。

条件顺序在这里也是业务规则。如果一条订单既已取消又是负金额，因为 `cancelled` 先判断，它只会记录"已取消"。修改分支顺序就会改变可观察结果，不能只把这种调整看成格式重排。

Java 工程师常见迁移陷阱

容易写错或误判的地方	实际表现	更稳妥的做法
更宽泛的条件放在前面	后面的具体分支永远无法到达	从非法或具体条件逐步走向一般条件
用 `if value:` 区分 `None` 和 `0`	两者都会进入假值分支	业务含义不同时显式写 `is None`
只为取值使用 `range(len(items))`	多维护一层下标并增加越界风险	直接 `for item in items`
给循环变量重新赋值，期待修改列表	只改变名字绑定，原列表不变	构造新列表或按下标回写
忘记 `range` 不包含终点	少处理一个边界值	先手算起点、终点和步长
`while` 中没有推进状态	条件可能永远为真	明确初值、继续条件和推进动作
把 `pass` 当成 `continue`	本轮后续代码照常执行	跳过本轮必须写 `continue`
默认 `zip` 会检查长度	较长输入的尾部被静默丢弃	业务要求等长时用 `strict=True` 并实际消费

以为循环 `else` 表示“零次循环”	正常耗尽都会执行 `else`	把它读成“没有 `break`”
在推导式中写日志和状态更新	产生无用结果且难以调试	有副作用或多步流程就用普通循环
遍历时增删原列表或字典键	跳过数据或抛运行时错误	构造新容器，或遍历明确的副本

小练习

1. **条件边界**：实现 `classify_score(score)`。`90~100` 为“优秀”，`60~89` 为“及格”，`0~59` 为“不及格”，区间外返回“非法分数”。测试 `-1`、`0`、`59`、`60`、`89`、`90`、`100`、`101`。
2. **整数序列**：分别用 `range` 生成 `0~10` 的偶数和从 `10` 到 `1` 的倒计时。解释每个起点、终点和步长为什么这样填写。
3. **组合遍历**：并行遍历商品名和价格，打印 `1. 商品名: 价格`。故意让两组数据长度不同，比较默认 `zip` 与 `strict=True` 的行为。
4. **搜索流程**：跳过空用户名；遇到第一个长度大于 12 的名字就停止，否则在循环正常结束后打印“没有过长用户名”。
5. **选择表达方式**：从订单金额中筛出 `>= 100` 的合法金额并计算含税金额。先写推导式，再加入非法负数日志并改成普通循环，比较哪种形式更能表达需求。
6. **综合练习**：为订单加入“已取消”状态。取消订单应优先跳过，金额非法时记录原因；最后输出成功订单数、总额和平均金额，并处理没有成功订单的情况。

做练习时先列边界输入，再写正常路径。控制流是否可靠，往往由空输入、临界值、找不到目标和长度不一致这些情况决定。

运行本章示例

本章只使用 Python 标准库，不需要安装额外依赖。仓库要求 Python 3.14 或更高版本。

在仓库根目录运行：

```
.venv/bin/python -B java_to_python/chapters/06_control_flow/example.py
```

如果已经激活仓库虚拟环境，也可以运行：

```
python3 -B java_to_python/chapters/06_control_flow/example.py
```

配套文件的职责是：

- `example.py`：可直接运行的 Python 示例，包含边界数据与练习结果；

关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下

4. 搜索流程：跳过空用户名，遇到第一个长度大于 12 的名字就停止，否则让循环正常结束后打印“没有过长用户名”。
5. 选择表达方式：从订单金额中筛出 `>= 100` 的合法金额并计算含税金额。先写推导式，再加入非法负数日志并改成普通循环，比较哪种形式更能表达需求。
6. 综合练习：为订单加入“已取消”状态。取消订单应优先跳过，金额非法时记录原因；最后输出成功订单数、总额和平均金额，并处理没有成功订单的情况。

做练习时先列边界输入，再写正常路径。控制流是否可靠，往往由空输入、临界值、找不到目标和长度不一致这些情况决定。

运行本章示例

本章只使用 Python 标准库，不需要安装额外依赖。仓库要求 Python 3.14 或更高版本。

在仓库根目录运行：

```
.venv/bin/python -B java_to_python/chapters/06_control_flow/example.py
```

如果已经激活仓库虚拟环境，也可以运行：

```
python3 -B java_to_python/chapters/06_control_flow/example.py
```

配套文件的职责是：

- `example.py`：可直接运行的 Python 示例，包含边界数据与练习结果；
- `Chapter06ControlFlow.java`：用 Java 展示相近问题，便于比较循环、下标和集合构造方式。

运行 Python 示例时重点观察：宽泛和严格分支的匹配顺序、`range` 的终点、默认 `zip` 的截断、`strict=True` 的异常、`pass` 后仍执行的语句、循环 `else` 的进入条件，以及取消订单优先于金额校验的结果。

本章小结

Python 控制流的核心可以压缩成四句话：

1. 分支按顺序匹配，缩进决定归属，真值规则决定条件是否成立。
2. `for` 直接消费可迭代对象；只有确实需要整数或位置时才引入 `range` 和下标。
3. `continue` 结束本轮，`break` 结束最近一层循环，`pass` 什么也不结束，循环 `else` 表示没有通过 `break` 离开。
4. 推导式适合简单、无副作用的结果构造；一旦流程包含状态、日志或多种退出原因，普通循环通常更可维护。

```python
"""第 6 章：控制流。

直接运行本文件，观察条件分支、循环控制、range、enumerate、zip 和推导式。
每组输出都对应 README.md 中的一个小节。
"""

from dataclasses import dataclass, fields


@dataclass
class LoopDemoRunState:
    """只为演示 fields(...) 返回的字段怎样被 for 遍历。"""

    run_id: str
    phase: str = "created"
    attempts: int = 0


def show_section(title: str) -> None:
    """为每组示例打印醒目的标题，方便把代码与输出对应起来。"""
    print(f"\n{'=' * 12} {title} {'=' * 12}")


def classify_temperature(temperature: int) -> str:
    """按从严格到宽松的顺序判断温度所属区间。"""
    if temperature >= 35:
        return "高温预警"
    if temperature >= 26:
        return "偏热"
    if temperature >= 18:
        return "舒适"
    return "偏冷"


def classify_score(score: int) -> str:
    """按边界完整、互不重叠的区间给成绩分类。"""
    if not 0 <= score <= 100:
```

```python

def classify_score(score: int) -> str:
    """按边界完整、互不重叠的区间给成绩分类。"""
    if not 0 <= score <= 100:
        return "非法分数"
    if score >= 90:
        return "优秀"
    if score >= 60:
        return "及格"
    return "不及格"


def find_first_even(values: list[int]) -> int | None:
    """返回第一个偶数；遍历结束仍未返回时，结果为 None。"""
    for value in values:
        if value % 2 == 0:
            return value
    return None


def find_product(products: list[str], target: str) -> int | None:
    """用 for...else 查找商品，并返回位置。

    只有循环正常耗尽、从未执行 break，else 才会运行。
    """
    for index, product in enumerate(products):
        if product == target:
            print(f"找到 {target}，准备结束搜索")
            break
    else:
        print(f"没有找到 {target}；循环是正常耗尽的")
        return None

    # 只有 break 路径能走到这里，所以 index 一定已经绑定。
    return index


```

```python
def find_first_overlong_username(usernames: list[str]) -> str | None:
    """跳过空名字，找到第一个长度大于 12 的名字后停止。"""
    found: str | None = None
    for username in usernames:
        if not username:
            continue
        if len(username) > 12:
            found = username
            break
    else:
        print("  没有过长用户名")
    return found


def demonstrate_conditionals() -> None:
    show_section("6.1 条件分支与真值")

    for temperature in [38, 29, 22, 10]:
        print(temperature, "度 ->", classify_temperature(temperature))

    # Python 会把空容器、空字符串、0 和 None 当作假值。
    cart: list[str] = []
    if not cart:
        print("购物车为空")

    # and 和 or 会短路：前半部分已能确定结果时，后半部分不会执行。
    user: dict[str, str] | None = None
    if user is not None and user.get("name"):
        print("用户名:", user["name"])
    else:
        print("没有可显示的用户名，且没有对 None 调用 get")

    age = 20
    print("18 <= age < 60;", 18 <= age < 60)

    temperature = 27
```

```python
def demonstrate_conditionals() -> None:

    temperature = 27
    feeling: Literal['热'] | Literal['舒适'] = "热" if temperature >= 26 else "舒适"
    print("条件表达式:", feeling)


def demonstrate_for_and_range() -> None:
    show_section("6.2 for 与 range")

    languages: list[str] = ["Java", "Python", "Go"]
    print("只需要元素时，直接迭代:")
    for language in languages:
        print("-", language)

    odd_numbers = range(1, 8, 2)
    print("range 对象:", odd_numbers)
    print("起点包含、终点不包含:", list(odd_numbers))
    print("反向计数:", list(range(5, 0, -1)))
    print("方向与步长不匹配会得到空序列:", list(range(1, 5, -1)))


def demonstrate_how_to_read_for_forms() -> None:
    show_section("6.2.1 普通 for、列表推导式与 lambda")

    # fields(...) 返回一组字段对象。先不用关心 dataclass 的细节，只把它当成
    # “能逐个取出元素的数据源”；第 14 章会专门解释 fields。
    state_fields: tuple[Field[Any], ...] = fields(LoopDemoRunState)

    # 写法 1：普通 for 语句。它可以在每轮执行多行代码，也可以打印日志或提前停止。
    field_names_by_loop: list[str] = []
    for item in state_fields:
        field_name: str = item.name
        field_names_by_loop.append(field_name)

    # 写法 2：列表推导式。阅读时先看中间的 for，再看最左边要放入列表的值：
    # for item in state_fields -> 每次取一个字段
```

```python
def demonstrate_how_to_read_for_forms() -> None:
    # 写法 2：列表推导式。阅读时先看中间的 for，再看最左边要放入列表的值：
    # for item in state_fields -> 每次取一个字段
    # item.name                -> 把这个字段的 name 放入新列表
    field_names_by_comprehension: list[str] = [item.name for item in state_fields]
    print("普通 for 得到字段名:", field_names_by_loop)
    print("列表推导式得到字段名:", field_names_by_comprehension)
    print("两种写法结果相同:", field_names_by_loop == field_names_by_comprehension)

    # 写法 3：lambda 只定义“给我一个参数，我返回什么”，它本身不会遍历。
    # 真正逐个遍历 state_fields 的是 map；list 再把 map 的惰性结果收集起来。
    field_names_by_map: list[str] = list(  # noqa: C417  # 教学对照：这里故意展示 lambda + map
        map(lambda item: item.name, state_fields)
    )
    print("lambda + map 得到字段名:", field_names_by_map)

    # for 左边可以一次接住多个值，这叫解包，不是另一种循环。
    course_results: list[tuple[str, int]] = [("控制流", 90), ("函数", 95)]
    for course_name, score in course_results:
        print(f"元组解包 -> {course_name}: {score}")

    # 遍历字典本身只能得到键；items() 每轮提供一个（键，值）元组，再由左边解包。
    settings: dict[str, int] = {"timeout": 3, "retries": 2}
    for setting_name, setting_value in settings.items():
        print(f"字典 items() -> {setting_name}={setting_value}")


def demonstrate_enumerate_and_zip() -> None:
    show_section("6.3 enumerate 与 zip")

    languages: list[str] = ["Java", "Python", "Go"]
    scores: list[int] = [90, 95, 88]

    # start=1 只改变产生的编号，不会改变原列表。
    for display_number, language in enumerate(languages, start=1):
        print(f"{display_number}. {language}")
```

```
165  def demonstrate_enumerate_and_zip() -> None:
171      # start=1 只改变产生的编号，不会改变原列表。
172      for display_number, language in enumerate(languages, start=1):
173          print(f"{display_number}. {language}")
174
175      for language, score in zip(languages, scores, strict=True):
176          print(f"{language} -> {score}")
177
178      # 默认 zip 会在较短输入耗尽时结束，末尾的 Python 会被静默丢掉。
179      default_pairs: list[tuple[str, int]] = list(zip(["Java", "Python"], [90]))
180      print("默认 zip 的静默截断:", default_pairs)
181
182      # 教学反例放在窄范围异常捕获中，不会中断后续示例。
183      try:
184          list(zip(["Java", "Python"], [90], strict=True))
185      except ValueError as error:
186          print("strict=True 发现长度不一致:", error)
187
188
189  def demonstrate_while() -> None:
190      show_section("6.4 while 与终止条件")
191
192      # 用确定性的布尔列表模拟三次请求，不依赖网络或随机数。
193      request_results: list[bool] = [False, False, True]
194      attempt = 0
195      max_attempts: int = len(request_results)
196
197      while attempt < max_attempts:
198          succeeded: bool = request_results[attempt]
199          attempt += 1  # 每轮推进状态；漏掉它可能造成死循环。
200          print(f"第 {attempt} 次尝试:", "成功" if succeeded else "失败")
201          if succeeded:
202              break
203      else:
204          # 如果所有尝试都失败，循环会正常耗尽并进入这里。
205          print("已达到最大尝试次数")
206
```

```python
def demonstrate_loop_control() -> None:
    show_section("6.5 break、continue、pass 与循环 else")

    raw_scores: list[int] = [95, -1, 82, 101, 60]
    valid_scores: list[int] = []
    for score in raw_scores:
        if not 0 <= score <= 100:
            print("跳过非法分数:", score)
            continue  # 直接进入下一轮，下面的 append 不会执行。
        valid_scores.append(score)
    print("合法分数:", valid_scores)

    products: list[str] = ["键盘", "鼠标", "显示器"]
    print("搜索鼠标的位置:", find_product(products, "鼠标"))
    print("搜索音箱的位置:", find_product(products, "音箱"))

    # pass 只是什么都不做，并不会跳过本轮；它常用作暂时的语法占位符。
    for note in ["待处理", "", "已完成"]:
        if not note:
            pass
        print("pass 后仍会执行本轮后续代码:", repr(note))


def demonstrate_comprehensions() -> None:
    show_section("6.6 推导式与生成器表达式")

    numbers = range(1, 11)
    even_squares: list[int] = [number**2 for number in numbers if number % 2 == 0]
    print("偶数平方列表:", even_squares)

    # 字典的键重复时，后计算出的值会覆盖旧值。
    parity: dict[int, str] = {number: "偶数" if number % 2 == 0 else "奇数" for number in range(1, 6)}
    print("奇偶字典:", parity)

    languages: list[str] = ["Java", "Python", "JavaScript", "Go"]
    unique_initials: set[str] = {language[0] for language in languages}
```

```
31  def demonstrate_comprehensions() -> None:
41
42      languages: list[str] = ["Java", "Python", "JavaScript", "Go"]
43      unique_initials: set[str] = {language[0] for language in languages}
44      print("首字母集合:", sorted(unique_initials))
245
246     # 生成器表达式不会先创建完整列表；sum 取值时才逐个计算。
247     total: int = sum(number**2 for number in range(1, 5))
248     print("生成器表达式求平方和:", total)
249
250     # 有多步业务规则或需要日志时，普通循环通常比推导式更清楚。
251     taxable_orders: list[float] = [120.0, -10.0, 80.0, 200.0]
252     taxed_orders: list[float] = []
253     for amount in taxable_orders:
254         if amount < 0:
255             print("忽略非法订单金额:", amount)
256             continue
257         if amount >= 100:
258             taxed_orders.append(round(amount * 1.13, 2))
259     print("大额订单含税金额:", taxed_orders)
260
261
262 def demonstrate_readme_exercises() -> None:
263     """运行 README 中控制流相关的五组独立练习。"""
264     show_section("README 小练习 1~5")
265
266     print("练习 1：成绩边界")
267     for score in (-1, 0, 59, 60, 89, 90, 100, 101):
268         print(f"  {score:>3} -> {classify_score(score)}")
269
270     print("练习 2：range 的起点、终点和步长")
271     print("  0~10 偶数 range(0, 11, 2):", list(range(0, 11, 2)))
272     print("  10~1 倒计时 range(10, 0, -1):", list(range(10, 0, -1)))
273
274     print("练习 3：enumerate + zip")
275     products: list[str] = ["键盘", "鼠标", "显示器"]
276     prices: list[int] = [299, 99, 1_299]
```

```
262  def demonstrate_readme_exercises() -> None:
276      prices: list[int] = [299, 99, 1_299]
277      for number, (product, price) in enumerate(
278          zip(products, prices, strict=True),
279          start=1,
280      ):
281          print(f"  {number}. {product}: ¥{price}")
282
283      short_prices: list[int] = [299, 99]
284      print("  默认 zip 静默截断:", list(zip(products, short_prices)))
285      try:
286          list(zip(products, short_prices, strict=True))
287      except ValueError as error:
288          print("  strict=True 拒绝长度不一致:", error)
289
290      print("练习 4: continue、break 与循环 else")
291      first_result: str | None = find_first_overlong_username(["Alice", "", "very_long_username", "Bob"])
292      print("  第一个过长用户名:", first_result)
293      second_result: str | None = find_first_overlong_username(["Alice", "", "Bob"])
294      print("  正常耗尽后的结果:", second_result)
295
296      print("练习 5: 含税金额的推导式与普通循环")
297      amounts: list[float] = [120.0, -10.0, 80.0, 200.0]
298      comprehension_result: list[float] = [round(amount * 1.13, 2) for amount in amounts if amount >= 100]
299      print("  推导式结果:", comprehension_result)
300
301      loop_result: list[float] = []
302      for amount in amounts:
303          if amount < 0:
304              print("  普通循环记录非法负数:", amount)
305              continue
306          if amount >= 100:
307              loop_result.append(round(amount * 1.13, 2))
308      print("  普通循环结果:", loop_result)
309
310
311  def demonstrate_combined_case() -> None:
```

```python
def demonstrate_combined_case() -> None:
    show_section("6.7 与 README 练习 6：处理订单")

    # 每条元组依次表示订单号、金额、是否已支付、是否已取消。
    orders: list[tuple[str, float, bool, bool]] = [
        ("A100", 120.0, True, False),
        ("A101", -20.0, True, False),
        ("A102", 80.0, False, False),
        ("A103", 200.0, True, False),
        ("A104", -50.0, True, True),
    ]
    completed: list[tuple[str, float]] = []

    for order_id, amount, paid, cancelled in orders:
        # 取消状态优先判断，因此 A104 会记录“已取消”，而不是“金额非法”。
        if cancelled:
            print(f"{order_id}: 已取消，跳过")
            continue
        if amount <= 0:
            print(f"{order_id}: 金额非法，跳过")
            continue
        if not paid:
            print(f"{order_id}: 尚未支付，跳过")
            continue

        level: Literal['大额'] | Literal['普通'] = "大额" if amount >= 100 else "普通"
        completed.append((order_id, amount))
        print(f"{order_id}: {level}订单处理完成")

    completed_ids: list[str] = [order_id for order_id, _ in completed]
    total_amount: float | Literal[0] = sum(amount for _, amount in completed)
    completed_count: int = len(completed)
    average_amount: float | None = total_amount / completed_count if completed else None
    print("完成的订单:", completed_ids)
    print("完成订单数:", completed_count)
    print("完成订单总额:", total_amount)
    print("完成订单平均金额:", average_amount)
```

```
311  def demonstrate_combined_case() -> None:
345      print("完成订单数:", completed_count)
346      print("完成订单总额:", total_amount)
347      print("完成订单平均金额:", average_amount)
348
349      no_completed_orders: list[tuple[str, float]] = []
350      empty_average: float | None = (
351          sum(amount for _, amount in no_completed_orders) / len(no_completed_orders)
352          if no_completed_orders
353          else None
354      )
355      print("没有成功订单时的平均金额:", empty_average)
356
357
358  def main() -> None:
359      demonstrate_conditionals()
360      demonstrate_for_and_range()
361      demonstrate_how_to_read_for_forms()
362      demonstrate_enumerate_and_zip()
363      demonstrate_while()
364      demonstrate_loop_control()
365      demonstrate_comprehensions()
366      demonstrate_readme_exercises()
367      demonstrate_combined_case()
368      print("\n找到的首个偶数:", find_first_even([1, 3, 8, 10]))
369      print("没有偶数时:", find_first_even([1, 3, 5]))
370
371
372  if __name__ == "__main__":
373      main()
374
```